

CyberLab Deployment Guide

Sep 26, 2026 · @Saaaid

One segmented Internet/DMZ/LAN lab with Suricata and Portainer, deployed six ways from the same scripts: a VirtualBox Kali VM (free), AWS Academy Learner Lab (free with class credits), Amazon Lightsail (about \$44/month, or less if parked between sessions), a low-cost KVM VPS (about \$15-30/month), Hetzner Cloud (about €9/month), or an institution's own Proxmox server (one VM per student, no per-student cost).

About this guide

This guide lets any student, instructor or institution deploy the same CyberLab: a segmented Internet/DMZ/LAN cyber range with network intrusion detection, built for project-based cybersecurity courses and mapped to the NICE Framework.

You are	Read	Typical option
A student with a capable laptop	Student quick start (second tab), then Part 1	1. VirtualBox
A student whose course uses AWS Academy	Student quick start, then Part 2	2. Learner Lab
An instructor running labs for a class	This tab: The lab and the kit, Parts 1 to 6, Using the lab, Adopting CyberLab	2, 3, 5 or 6
A campus IT or range administrator	Part 6 and Adopting CyberLab; for a multi-node cluster, the reference build below	6. Proxmox

Version, status and license

Item	Value
Version	0.9 (pilot), September 2026

Item	Value
Status	Scripts pass syntax and lint checks and the lab file validates; not yet run end to end on every option. Pilot on one machine before a class.
Maintainer	Saaïd Elhadad (add your institution and a public contact address before sharing)
License	To be chosen by the maintainer before public release; a common choice is CC BY 4.0 for this guide and MIT for the scripts
Reference build	On-Prem Cyber Range Runbook (Proxmox): a full 3-node cluster with Netgate, VLANs, Guacamole and OpenVPN

Get the kit

Everything in this guide comes from one archive, `cyberlab-kit.zip`, published by the maintainer (for example as a GitHub release; add the link here). After downloading, check that nothing was altered in transit:

```
unzip cyberlab-kit.zip && cd cyberlab-kit
sha256sum -c SHA256SUMS           # every line should end in OK
```

On Windows, run the same commands in WSL or Git Bash, or compare `Get-FileHash <file> -Algorithm SHA256` against `SHA256SUMS`.

Terms of use

- The lab contains intentionally vulnerable software. Run it only on machines and accounts you own or are authorized to use, and never expose its targets to the internet or to a production network.
- Attack only the targets inside your own lab. Scanning or attacking anything else, including other students' labs or campus systems, is outside the purpose of this guide and may break the law and your institution's policies.
- Institutions adopting the lab should publish an acceptable use policy and have students accept it before they get access (see Adopting CyberLab).

Choosing a hosting option

Start with VirtualBox if the student's computer has 8 GB of RAM or more; otherwise use the Learner Lab when the course has it, and Hetzner Cloud, the cheapest paid option, when it does not.

Option	Cost	Host size	Best for	Main limits
1. VirtualBox (local)	Free	Kali VM with 6 GB RAM, 2 CPUs, 60 GB disk (laptop with 8 GB+ RAM)	Students with a capable laptop	x86 laptops only; Apple Silicon Macs run the x86 targets slowly or not at all
2. AWS Academy Learner Lab	Free to students; uses class credits (t3.large about \$0.08/hour)	EC2 t3.large, 2 vCPU, 8 GB RAM, 40 GB gp3	Courses already enrolled in AWS Academy	us-east-1/us-west-2 only, instances large or smaller, no Marketplace AMIs, instance stops when the session ends
3. Amazon Lightsail	\$44/month (8 GB) or \$24/month (4 GB + swap), billed hourly	2 vCPU, 8 GB RAM, 160 GB SSD	Instructor-run class instance or a student who wants a fixed price	A stopped instance still bills; park it (snapshot + delete) between sessions
4. Low-cost KVM VPS	Hostinger KVM 4 (16 GB) is \$14.99/month in year one on an annual plan, \$30.99 after	8-16 GB RAM, KVM plan only	A whole semester at the lowest monthly price	Annual prepay for the low price; renewal costs jump; no hourly billing

Option	Cost	Host size	Best for	Main limits
5. Hetzner Cloud	CX33 €8.49/month + €0.50 IPv4, billed hourly (€0.0136/hour) with a monthly cap	CX33: 4 vCPU (x86), 8 GB RAM, 80 GB NVMe	The cheapest always-on cloud host; one server per student for about €9/month	CX plans only in Germany and Finland (about 100 ms from the US East Coast); never pick an Arm CAX plan; new accounts may need ID verification
6. On-prem Proxmox cluster	No per-student cost once the hardware is in place (power and staff time only)	One VM per student: 2 vCPU, 6 GiB RAM, 40 GiB thin disk; about 38 pods per 256 GiB node	Whole classes with Guacamole or VPN access, recordings and central resets	Needs the cluster, switch and Netgate build; pods have no internet during term, so the template must be complete

The lab uses about 4-4.5 GB of RAM with every service running, so 8 GB hosts are comfortable and 4 GB hosts need the 4 GB swap file the cloud bootstrap adds.

The lab and the kit

Every option runs the same Docker Compose lab on one Linux host; only the way you get that host differs.

```

Internet 10.10.0.0/24          DMZ 10.20.0.0/24          LAN
10.30.0.0/24
+-----+          +-----+          +-----+
-----+
| attacker  .10      |--router--| juiceshop  .11:3000 |          |
metasploitable .20  |
| (Kali tools)      | .254  | dvwa        .12:80  |=====| (reached
only via          |
+-----+          fw:80,  +-----+          dvwa    | the DVWA
pivot)          |
                |          3000,  (no internet access)  .12  +-----+
-----+
    Suricata sensor      ICMP          Suricata
sensor
on br-inet              on br-lan

```

Service	Network	Purpose
attacker (Kali)	Internet	Student attack shell with nmap, sqlmap, hydra, netcat
router (Alpine)	Internet + DMZ	Firewall: only TCP 80/3000 and ICMP from the Internet into the DMZ
juiceshop, dvwa	DMZ	Web targets; DVWA is also on the LAN and is the pivot host
metasploitable	LAN	Internal target, unreachable from the Internet
suricata	Host	IDS on the perimeter and LAN bridges, ET Open rules
portainer	Management	Docker GUI at https://localhost:9443, host only
socks	Shares attacker's network	Optional SOCKS5 proxy at 127.0.0.1:1080 so a browser can reach the DMZ

Files in the kit (cyberlab-kit.zip)

File	Used by	What it does
bootstrap-local.sh	Part 1	Installs Docker, writes the lab to /opt/cyberlab, starts it
bootstrap-cloud.sh	Parts 2-4	Same, plus firewall (SSH only), fail2ban, auto-updates, swap; works as EC2/Lightsail user-data
lab/docker-compose.yml	All	The lab itself
scripts/cyberlab	All	Lab manager: up, verify, status, alerts, rules, reset, down, destroy
virtualbox/vbox-tune-kali.ps1 / .sh	Part 1	Sets VM RAM/CPU/NAT and takes a clean snapshot
aws/learner-lab-launch.sh, learner-lab-ip.sh	Part 2	Launch the EC2 instance; restart it and get the new IP after a session
aws/lightsail-launch.sh, lightsail-park.sh	Part 3	Launch the instance; park and restore it between sessions
vps/vps-first-login.sh	Part 4	Creates a sudo user, copies the SSH key, runs the bootstrap
build/build-bootstrap.py	Instructor	Regenerates both bootstrap files after you edit the lab or scripts
hetzner/hcloud-launch.sh, hcloud-park.sh	Part 5	Launch a CX33 with a cloud firewall and a labadmin user; park and restore it
proxmox/ (range.conf, pod-*.sh, template-*.sh, single-host-network.sh, guacamole/, guac-provision.py)	Part 6	Build the offline pod template, create and manage one VM per student, isolate the range network, run Guacamole
SHA256SUMS	All	Checksums of every file; run sha256sum -c SHA256SUMS after download

The full text of every script is in [Scripts](#).

Steps every option shares

1. Get a Linux host (Kali VM, EC2, Lightsail or VPS).
2. Copy the bootstrap file to it, or pass it as user-data.
3. Run it with sudo and wait 10-15 minutes. Progress goes to `/var/log/cyberlab-bootstrap.log`.
4. Run `cyberlab verify` and confirm every check shows PASS.
5. Take a snapshot of the finished host so you can roll back.

Part 1: VirtualBox (local)

A student with an x86 laptop and 8 GB+ RAM runs the whole lab free inside one Kali VM in about 30 minutes.

Requirements: Windows 10/11, Linux or an Intel Mac; 8 GB RAM (16 GB is better); 60 GB free disk; hardware virtualization (Intel VT-x or AMD-V) turned on in the BIOS/UEFI.

1. **Install VirtualBox** from [virtualbox.org](https://www.virtualbox.org), plus 7-Zip on Windows.
2. **Download Kali:** kali.org/get-kali → Virtual Machines → VirtualBox 64-bit (.7z). Extract it.
3. **Import the VM:** double-click the extracted `.vbox` file, or use VirtualBox → Machine → Add. Do not start it yet.
4. **Tune the VM** (it must be powered off). List names with `VBoxManage list vms`, then run one of these:

```
# Windows PowerShell
.\vbox-tune-kali.ps1 -VmName "kali-linux-2026.3-virtualbox-amd64"
```

```
# macOS (Intel) or Linux
bash vbox-tune-kali.sh "kali-linux-2026.3-virtualbox-amd64"
```

This sets 6 GB RAM, 2 CPUs and NAT networking, and takes a snapshot called clean-before-cyberlab. On an 8 GB laptop, use `-RamMB 5120`.

5. **Start Kali** and log in (default user kali, password kali). Change the password with `passwd`.
6. **Get the kit into the VM:** download `cyberlab-kit.zip` in Kali's Firefox from wherever you post it (LMS, GitHub), or use a VirtualBox shared folder.
7. **Run the bootstrap:**

```
cd ~/Downloads && unzip cyberlab-kit.zip && cd cyberlab-kit
sudo bash bootstrap-local.sh kali
```

The first run downloads about 3 GB of images and installs the attacker tools, so expect 10-20 minutes.

8. **Log out and back in** so the kali user can run docker without sudo.
9. **Verify:** `cyberlab verify` should show PASS for every check.
10. **Snapshot again:** power off and take a snapshot called cyberlab-ready. Restoring it is the fastest full reset.

Browser access inside Kali: Portainer is at <https://localhost:9443>. To browse the DMZ as the attacker, set Firefox → Settings → Network Settings → Manual proxy → SOCKS Host 127.0.0.1, port 1080, SOCKS v5, then open <http://10.20.0.11:3000> (Juice Shop) or <http://10.20.0.12> (DVWA).

If the laptop struggles: stop Portainer (`docker stop portainer`) to save about 250 MB, or move the student to Part 2.

Part 2: AWS Academy Learner Lab

Each student launches one t3.large Ubuntu instance with the bootstrap as user-data; it costs about \$0.08 per running hour from the class credits and nothing from the student.

Learner Lab rules that shape this setup: only us-east-1 and us-west-2, instances up to "large", Amazon-provided AMIs only (so no Kali Marketplace image; the Kali tools come from the attacker container), EBS up to 100 GB, and running instances are stopped when the session ends.

A. Start the lab

1. In the AWS Academy course, open **Learner Lab** and click **Start Lab**. Wait until the dot next to AWS turns green.
2. Click **AWS Details** → **Download PEM** to save labsuser.pem (the vockey private key).
3. Click **AWS** to open the console. Confirm the region is **N. Virginia (us-east-1)**, because vockey exists only there.

B. Launch the instance (console method)

1. EC2 → **Launch instance**. Name: `cyberlab`.

2. AMI: **Ubuntu Server 24.04 LTS** (Quick Start tab).
3. Instance type: **t3.large** (2 vCPU, 8 GB).
4. Key pair: **vockey**.
5. Network settings → Create security group → **Allow SSH from My IP**. Leave HTTP and HTTPS unchecked.
6. Storage: **40 GiB gp3**.
7. Advanced details → IAM instance profile: **LabInstanceProfile**.
8. Advanced details → User data: paste the whole content of bootstrap-cloud.sh (14 KB, under the 16 KB limit).
9. **Launch instance**, then copy its public IPv4 address.

B (alternative). Launch with the AWS CLI

On your own computer with the AWS CLI installed, click **AWS Details** → **Show** next to AWS CLI and paste the block into `~/.aws/credentials` (it expires when the session ends). Then:

```
cd cyberlab-kit
bash aws/learner-lab-launch.sh bootstrap-cloud.sh
```

The script finds the current Ubuntu 24.04 AMI, creates an SSH-only security group for your IP, and launches t3.large with LabInstanceProfile and the bootstrap.

C. Connect and verify

```
# macOS / Linux
chmod 400 labsuser.pem
ssh -i labsuser.pem -L 9443:127.0.0.1:9443 -L 1080:127.0.0.1:1080
ubuntu@<public-ip>
```

```
# Windows PowerShell (built-in OpenSSH)
icacls labsuser.pem /inheritance:r /grant:r "$($env:USERNAME):(R)"
ssh -i labsuser.pem -L 9443:127.0.0.1:9443 -L 1080:127.0.0.1:1080
ubuntu@<public-ip>
```

On the instance, run `tail -f /var/log/cyberlab-bootstrap.log` until it says done (10-15 minutes), then `cyberlab verify`. While the SSH session is open, Portainer is at `https://localhost:9443` on your own laptop, and the SOCKS proxy is at `127.0.0.1:1080`.

D. Each later session

1. **Start Lab**, then EC2 → select cyberlab → **Start instance** (or run `bash aws/learner-lab-ip.sh`, which also re-allows SSH from your current IP).
2. The public IP changes after every stop, so copy the new one. The containers restart by themselves.
3. When finished, **Stop** the instance (do not terminate it) and click **End Lab**. A stopped instance only uses credits for its 40 GB disk, about \$3.20 a month.

If SSH times out after moving between home and campus, your IP changed: edit the security group's SSH rule to **My IP**, or rerun `learner-lab-ip.sh`.

Part 3: Amazon Lightsail

Use the \$44/month plan (2 vCPU, 8 GB, 160 GB SSD); because Lightsail bills by the hour, parking the instance between sessions brings a twice-a-week, 3-hour lab down to a few dollars a month.

A. Launch (console method)

1. `lightsail.aws.amazon.com` → **Create instance**.
2. Region: the one closest to your students (for example Virginia, us-east-1).
3. Platform **Linux/Unix** → **OS Only** → **Ubuntu 24.04 LTS**.
4. **Add launch script**: paste the whole content of `bootstrap-cloud.sh`.
5. **Change SSH key pair** → Create new → download the .pem file and keep it safe.
6. Plan: **\$44 USD** (8 GB). The \$24 plan (4 GB) also works because the bootstrap adds 4 GB of swap, but it is noticeably slower.
7. Name it `cyberlab` and click **Create instance**.
8. Instance → **Networking** tab:
 - Delete the HTTP (80) rule.
 - Edit the SSH rule → **Restrict to IP address** → your public IP.
 - **Attach static IP** so the address stays the same after restarts (free while attached).

A (alternative). Launch with the AWS CLI

```
cd cyberlab-kit
bash aws/lightsail-launch.sh 8 bootstrap-cloud.sh # use 4 for the $24 plan
```

The script looks up the current bundle and Ubuntu blueprint IDs, creates and saves `cyberlab-key.pem`, launches the instance, replaces the default firewall with SSH from your IP only, and attaches a static IP.

B. Connect and verify

```
chmod 400 cyberlab-key.pem
ssh -i cyberlab-key.pem -L 9443:127.0.0.1:9443 -L 1080:127.0.0.1:1080
ubuntu@<static-ip>
tail -f /var/log/cyberlab-bootstrap.log      # wait for "done"
cyberlab verify
```

Lightsail's browser SSH button works for commands, but it cannot forward ports, so use a terminal when you want Portainer or the SOCKS proxy.

C. Park between sessions (saves money)

A stopped Lightsail instance still bills the full plan price. Instead, snapshot and delete it after class, then restore it before the next class:

```
bash aws/lightsail-park.sh park      # snapshot, then delete the instance
bash aws/lightsail-park.sh restore   # recreate from the latest snapshot
```

Snapshots cost \$0.05 per GB-month. The static IP is kept while parked; an unattached static IP is billed by the hour, so release it if the instance will be parked for weeks.

Class model: one 8 GB instance per student is simplest. To share one bigger instance, give each student a separate host instead, because this lab uses fixed container names and IPs and runs one copy per host.

Part 4: Low-cost KVM VPS (Hostinger, Contabo)

A flat monthly KVM VPS with 8-16 GB RAM that stays on all semester, with no hourly meter to manage.

A. Choose and order the plan

1. Pick a **KVM** plan (OpenVZ and LXC plans cannot run Docker reliably) with at least 8 GB RAM and 100 GB disk. Hostinger KVM 2 (8 GB) is enough; KVM 4 (16 GB) is \$14.99/month in year one on an annual plan and \$30.99/month after, or \$42.99 month-to-month. Contabo sells similar KVM plans.

2. OS template: **Ubuntu 24.04** (plain OS, no control panel or app template).
3. During setup, **add your SSH public key** (create one first with `ssh-keygen -t ed25519` and paste the .pub file). Also set a strong root password.
4. If the provider has a firewall panel (Hostinger hPanel → VPS → Firewall), allow only TCP 22 inbound.

B. Run the first-login script

```
# from your computer
scp cyberlab-kit.zip root@<vps-ip>:
ssh root@<vps-ip>

# on the VPS
apt-get update && apt-get install -y unzip
unzip cyberlab-kit.zip && cd cyberlab-kit
bash vps/vps-first-login.sh labadmin bootstrap-cloud.sh
```

The script checks that the VPS is really KVM, creates the sudo user labadmin, copies root's SSH key to it, turns off root login, then runs the cloud bootstrap (Docker, firewall, fail2ban, swap if needed, lab start and self-test).

C. Connect from now on

```
ssh -L 9443:127.0.0.1:9443 -L 1080:127.0.0.1:1080 labadmin@<vps-ip>
cyberlab verify
```

Test the labadmin login in a second terminal before closing the root session, so a key problem cannot lock you out.

D. Keep it healthy

- Take a provider snapshot after the first successful `cyberlab verify` (check your plan's snapshot or backup option in the provider panel).
- Security updates install automatically; reboot monthly with `sudo reboot`, and the lab comes back by itself.
- Put a calendar reminder a month before renewal, because the renewal price is much higher than the first-term price.

Part 5: Hetzner Cloud

A Hetzner CX33 (4 vCPU, 8 GB, x86) runs the full lab for €8.49 a month plus €0.50 for its IPv4 address, about a fifth of the Lightsail price for the same RAM.

Key facts: billing is hourly (€0.0136/hour) but never exceeds the monthly price, a powered-off server is still billed, and CX plans are offered in Nuremberg (nbg1), Falkenstein (fsn1) and Helsinki (hel1) only. Hetzner's US locations offer only the pricier CPX plans. Latency from the US to Germany is about 100 ms, which is fine for SSH work.

A. Set up the account and project

1. Sign up at console.hetzner.cloud. New accounts may be asked for ID or a small prepayment before servers can be created, so do this a few days before class.
2. Create a project called `cyberlab`.
3. On your computer, create an SSH key if you do not have one: `ssh-keygen -t ed25519`.

B. Launch (console method)

1. Project → **Add Server**.
2. Location: **Nuremberg**, **Falkenstein** or **Helsinki**.
3. Image: **Ubuntu 24.04**.
4. Type: **Shared vCPU → x86 (Intel/AMD) → CX33**. Do not choose an Arm64 (CAX) type: the DVWA and Metasploitable images are x86-only.
5. Networking: keep **Public IPv4** (students need it for SSH).
6. SSH keys: **Add SSH key** and paste your `~/.ssh/id_ed25519.pub`.
7. Firewalls: **Create Firewall** with one inbound rule, TCP port 22 from your IP, and apply it to the server.
8. Cloud config: paste the content of `bootstrap-cloud.sh`. Hetzner images log in as root; the bootstrap runs fine as root, but the CLI method below also creates a labadmin user for you.
9. Name it `cyberlab` and click **Create & Buy now**.

B (alternative). Launch with the hcloud CLI

```
# once: install hcloud (brew install hcloud, or the GitHub release), then add
a project API token
hcloud context create cyberlab          # paste a Read & Write token from
Project → Security → API tokens

cd cyberlab-kit
bash hetzner/hcloud-launch.sh nbg1 bootstrap-cloud.sh
```

The script uploads your SSH key, creates a cloud firewall that allows SSH from your IP only, creates the sudo user labadmin with root login disabled, and launches a CX33 with the bootstrap. Set `HCLOUD_TYPE=cx43` for 16 GB.

C. Connect and verify

```
ssh -L 9443:127.0.0.1:9443 -L 1080:127.0.0.1:1080 labadmin@<server-ip>  #
console method: root@<server-ip>
tail -f /var/log/cyberlab-bootstrap.log      # wait for "done"
cyberlab verify
```

D. Between sessions

For weekly labs across a semester, leave the server running; the monthly cap keeps it at about €9. For long breaks, park it:

```
bash hetzner/hcloud-park.sh park      # shut down, snapshot, delete
bash hetzner/hcloud-park.sh restore   # recreate from the newest snapshot
```

Snapshots are billed per GB-month, so delete old ones with `hcloud image delete <id>`. A restored server gets a new IP address.

Part 6: On-premises Proxmox

An institution's own Proxmox VE server gives every student a pod VM that runs this same lab, reached through Guacamole in a browser (and optionally through the institution's VPN), with no per-student cost after the hardware. The steps below build the smallest complete setup: **one Proxmox host** with an isolated range network. For a multi-node cluster behind a dedicated firewall and range VLAN, follow the reference build (On-Prem Cyber Range Runbook) and use the same kit scripts.

```

Students --HTTPS--> Guacamole VM (campus NIC)
                      | vmbr1 10.40.0.10
        isolated range bridge vmbr1 10.40.0.0/23 (no internet, no route to campus)
                      |
        pod001 10.40.1.1   pod002 10.40.1.2   ...   each runs the full
CyberLab

```

Requirements

Item	Minimum	Notes
Hypervisor	Proxmox VE 8 or 9 on an x86 server with VT-x/AMD-V	Arm servers cannot run the x86 target images
RAM	24 GiB for the host + 6 GiB per concurrently running pod	256 GiB runs about 38 pods at once; stopped pods cost disk only
Disk	60 GB for the template and Guacamole + about 8 GB per pod (thin clones)	SSD or NVMe strongly recommended
Network	One uplink with internet for the template build and Guacamole	Pods themselves never get internet
Access server	One Ubuntu 24.04 VM for Guacamole: 4 vCPU, 8 GB RAM, 60 GB	Runs on the same host
People	One administrator comfortable with Linux and Proxmox	About one day for the first build

A. Prepare the Proxmox host

```
# on the Proxmox host, as root (replace pve1 with your node name)
apt-get update && apt-get install -y jq unzip python3-requests
ssh-keygen -t ed25519 -N '' -f /root/.ssh/id_ed25519      # skip if it exists
ssh-copy-id root@$(hostname)                            # the scripts SSH
to the node, even on one host
mkdir -p /root/range/{out,logs,rosters} && cd /root/range
# copy cyberlab-kit.zip here (scp from your workstation), then:
unzip -oq cyberlab-kit.zip && (cd cyberlab-kit && sha256sum -c --quiet
SHA256SUMS)
cp -r cyberlab-kit/proxmox/* /root/range/
chmod 700 /root/range/*.sh /root/range/guac-provision.py
```

B. Set your site values in range.conf

For a single host, change these lines in `/root/range/range.conf` (the file explains every value):

```
NODES=$(hostname)      # or write the node name, e.g. NODES=(pve1)
TPL_BASE=(9000)
RANGE_BRIDGE=vmbr1
RANGE_VLAN=
RANGE_GW=10.40.0.1
BUILD_BRIDGE=vmbr0     # the bridge with internet access
BUILD_VLAN=
BUILD_IPCONFIG="ip=dhcp" # or a fixed address, e.g.
"ip=192.168.1.50/24,gw=192.168.1.1"
GUAC_IP=10.40.0.10
```

C. Create the isolated range network

```
bash /root/range/single-host-network.sh
# optional: also let your VPN pool reach pod SSH
#   VPN_NET=10.8.0.0/24 bash /root/range/single-host-network.sh
```

The script adds bridge `vmbr1` (`10.40.0.1/23`, no physical port) and loads `nftables` rules at every boot so the range cannot reach the Proxmox host, your campus network or the internet. If you pass `VPN_NET`, your VPN server must route `10.40.0.0/23` to this host's campus address.

D. Build the pod template

```
/root/range/template-build.sh create
/root/range/template-build.sh provision <vm-ip>      # IP: the one set in
BUILD_IPCONFIG, or the printed MAC in your DHCP leases
/root/range/template-build.sh seal
/root/range/distribute-templates.sh                  # prints "nothing to
distribute" on one node
```

`provision` installs Docker and the lab, runs `cyberlab verify`, and bakes every image, Suricata rule and attacker tool into the template, because pods have no internet. Every check must PASS before you run `seal`.

E. Create the Guacamole VM

```
qm create 300 --name svc-guac --ostype l26 --machine q35 --cpu host \
  --cores 4 --memory 8192 --agent enabled=1 --onboot 1 \
  --scsihw virtio-scsi-single --serial0 socket --vga serial0 \
  --net0 virtio,bridge=vibr0 --net1 virtio,bridge=vibr1
qm set 300 --scsi0 local-lvm:0,import-from=/var/lib/vz/template/iso/noble-
server-cloudimg-amd64.img,discard=on,ssd=1
qm set 300 --ide2 local-lvm:cloudinit --boot order=scsi0 \
  --ciuser svcadmin --sshkeys /root/.ssh/id_ed25519.pub \
  --ipconfig0 ip=dhcp --ipconfig1 ip=10.40.0.10/23
qm disk resize 300 scsi0 60G
qm start 300
```

```
# then, from the Proxmox host (use the VM's campus address)
scp /root/range/cyberlab-kit.zip svcadmin@<guac-campus-ip>:
ssh svcadmin@<guac-campus-ip> 'sudo apt-get update -qq && sudo apt-get
install -y -qq unzip && unzip -oq cyberlab-kit.zip && sudo bash cyberlab-
kit/proxmox/guacamole/setup-guacamole.sh range.example.edu'
```

Finish in the Guacamole web UI as the script prints: create `rangeadmin`, enroll TOTP, then remove `guacadmin`. Give the VM a DNS name and your institution's TLS certificate (put `fullchain.pem` and `privkey.pem` in `/opt/guac/certs` and run `sudo docker compose -f /opt/guac/docker-compose.yml restart nginx`).

F. Create student pods

```
cat > /root/range/rosters/fall26.csv <<'EOF'
pod,username,password
1,jdoe,
2,asmith,
EOF
/root/range/pod-bulk.sh /root/range/rosters/fall26.csv      # blank
passwords are generated
GUAC_URL=https://<guac-campus-ip>/guacamole \
  /root/range/guac-provision.py /root/range/out/credentials-<date>.csv
```

Each pod takes 2 to 3 minutes: linked clone, first boot, `cyberlab verify`, clean snapshot. The credentials CSV holds every student's password, so store it encrypted and delete working copies.

G. Test before class

1. Build pod 250 for yourself: `pod-create.sh 250 testuser 'Test-Pod-2026'`, then provision it in Guacamole.
2. `pod-ctl.sh verify 250`: every check PASS.
3. From pod 250's lab host: `ping -c1 10.40.0.1`, `curl -m5 http://example.com` and a ping to a campus address must all fail.
4. `pod-ctl.sh lab-reset 250` completes without internet, and `verify` passes again.
5. `pod-ctl.sh reset 250` restores the original password and files.
6. Start as many pods as you expect at peak and watch host RAM stay under 85 percent.

H. Run the term

Task	Command
Start or stop a pod or all pods	<code>pod-ctl.sh start N</code> · <code>pod-ctl.sh stop all</code>
Weekly health check	<code>pod-ctl.sh verify all</code>
Student broke the lab	<code>pod-ctl.sh lab-reset N</code> (keeps their files), or they run <code>cyberlab reset</code>
Return a pod to day one	<code>pod-ctl.sh reset N</code>

Task	Command
Late student	one-line roster, <code>pod-bulk.sh</code> , then <code>guac-provision.py</code>
End of term	<code>guac-provision.py <csv> --delete</code> , then <code>podctl.sh destroy all</code>

Notes. Never run `cyberlab reset --full` inside a pod: it deletes the Suricata rules, which cannot be downloaded again offline. Students have sudo and Docker inside their own pod by design; the Proxmox firewall on each pod and the nftables rules on the host are the containment boundary. Guacamole session recordings are stored in `/opt/guac/record`; if you use them for research, get IRB approval and student consent first.

Using the lab

Once `cyberlab verify` passes, every option behaves the same.

Task	Command or address
Attack shell	<code>docker exec -it attacker bash</code>
Lab status	<code>cyberlab status</code>
Live IDS alerts	<code>cyberlab alerts</code> (Ctrl+C to stop)
Full IDS events (JSON)	<code>docker exec suricata tail -f /var/log/suricata/eve.json</code>
Update Suricata rules	<code>cyberlab rules</code>
Docker GUI	https://localhost:9443 (cloud: through the SSH tunnel)
Browse the DMZ as the attacker	Browser SOCKS5 proxy 127.0.0.1:1080, then http://10.20.0.11:3000 or http://10.20.0.12
Reset targets to clean state	<code>cyberlab reset</code> (keeps rules, logs, Portainer)
Reset everything	<code>cyberlab reset --full</code>
Stop the lab	<code>cyberlab down</code>
Remove lab and images	<code>cyberlab destroy</code>

Portainer first login: create the admin user within 5 minutes of the container starting. If the setup page says it timed out, run `docker restart portainer` and try again.

Suggested exercise flow: recon from the attacker (`nmap -sV 10.20.0.0/24`), exploit DVWA's command injection, pivot from DVWA to Metasploitable on 10.30.0.20, then switch roles and find each step in `cyberlab alerts` . Ask students why the DMZ bridge is not monitored (the router's NAT hides the attacker's IP there).

Atomic Red Team (optional)

Run atomics only inside the attacker container, never on the host, because each test executes real commands.

```
docker exec -it attacker bash
apt-get update && apt-get install -y powershell
pwsh
```

```
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing)
Install-AtomicRedTeam -getAtomics
Invoke-AtomicTest T1046 -ShowDetailsBrief      # Network Service Discovery
Invoke-AtomicTest T1046 -GetPrereqs
Invoke-AtomicTest T1046 -TestNumbers 1
```

Then compare the ATT&CK techniques you ran with what Suricata alerted on. Endpoint techniques will mostly go undetected, because this lab has network sensors only; that gap is a useful discussion point.

Adopting CyberLab at your institution

An institution can run CyberLab for a whole program by choosing one or two options from the table at the top, then working through this checklist before the first class.

Before the term

- ☐ Pick the option(s): VirtualBox for students with capable laptops, plus one hosted option (Learner Lab, Hetzner, Lightsail, VPS or Proxmox) for everyone else
- ☐ Get approval from campus IT and the information security office; share the Terms of use and the Safety rules in this guide
- ☐ Publish an acceptable use policy that students accept before access: attack only your own lab, no attempts against range or campus infrastructure, activity may be logged
- ☐ If you will record sessions or use lab data for research, get IRB review and a consent process first
- ☐ Pin image versions in lab/docker-compose.yml (replace `:latest` with the tags you tested) so every student gets the same lab
- ☐ Run the full option once yourself and pass `cyberlab verify`; for Proxmox, pass the Part 6 test list
- ☐ Budget: cloud options need a funding source or AWS Academy credits; Proxmox needs server RAM for peak concurrency (6 GiB per running pod)
- ☐ Accessibility: offer the Guacamole or cloud route to any student who cannot run VirtualBox

During the term

- ☐ Give students the Student quick start and the one option they should use

- ☐ Tell students how to reset (`cyberlab reset`) and whom to contact when it fails
- ☐ Stop or park cloud hosts between sessions (Parts 3 and 5); stop Proxmox pods outside class hours
- ☐ Run `cyberlab verify` weekly on instructor hosts, or `pod-ctl.sh verify all` on Proxmox

After the term

- ☐ Destroy student cloud hosts or pods and delete old snapshots
- ☐ Delete or archive credentials files and Guacamole users
- ☐ Report problems and fixes back to the maintainer so the next version of the kit improves

Mapping to the curriculum

Each lab stage maps to NICE Framework work roles: reconnaissance and web testing to Vulnerability Analysis, firewall policy to Cybersecurity Infrastructure Support, Suricata alerts and timelines to Defensive Cybersecurity and Incident Response, and container hardening to Systems Administration. The reference build's Appendix A has the full mapping and a sample 3-hour lab. Confirm role names and TKS statements against the current NICE Framework release before citing them in a syllabus.

Troubleshooting and safety

Most problems come from memory, the attacker's first-start install, or an SSH rule pinned to an old IP.

Symptom	Likely cause	Fix
<code>cyberlab up</code> waits a long time on attacker	First start installs Kali tools	Wait up to 15 minutes; watch <code>docker logs -f attacker</code>
Containers exit with code 137	Out of memory	Add RAM or swap, or <code>docker stop portainer juiceshop</code> for lighter exercises

Symptom	Likely cause	Fix
DVWA or Metasploitable stuck in Restarting (255); <code>docker logs</code> shows "exec format error"	ARM host (Apple Silicon, ARM VPS) running x86-only images: DVWA is amd64, Metasploitable 2 is 32-bit i386	Use an x86 host (Part 2, 3, 4 or 5; on Hetzner, CX not CAX), or turn on x86 emulation: see Running the lab on an ARM host, below
DVWA starts after emulation is added, but Metasploitable still shows exec format error	Only amd64 emulation is registered; the Metasploitable image is i386 and needs its own emulator	Register i386 as well (step 2 below), then <code>docker restart metasploitable</code>
apt says qemu-user-static has no installation candidate	Newer Ubuntu releases made it a virtual package, provided by qemu-user-binfmt	<code>sudo apt install -y qemu-user-binfmt</code> (not the -hwe variant unless the host runs the HWE kernel)
Exec format error comes back after a reboot or after <code>cyberlab destroy</code> and <code>up</code>	Emulation was registered for one session only, or the compose file has no platform lines	Make emulation persistent and pin the platforms (steps 3 and 4 below)
A target shows "Up Less than a second" right after a fix	The container may be caught in the middle of a restart loop	Wait 30 seconds and check that the restart count has stopped rising (step 5 below)
Suricata shows no alerts	Rules not loaded yet	<code>cyberlab rules</code> , then retry; confirm traffic with <code>sudo tcpdump -i br-inet</code>
Suricata exits at start	Lab bridges did not exist yet	<code>docker restart suricata</code> after the lab is up
SSH times out (cloud)	Your public IP changed	Update the SSH rule to your current IP
Browser gets no page through the SOCKS proxy	SSH tunnel closed, or proxy DNS on	Reconnect with <code>-L 1080:127.0.0.1:1080</code> ; use IP addresses, not names
SOCKS proxy stops after the attacker restarts	It shares the attacker's network stack	<code>docker restart socks</code>

Symptom	Likely cause	Fix
Portainer says the instance timed out	Admin user not created within 5 minutes	<code>docker restart portainer</code>
Bootstrap stopped part way	Network or apt error	Read <code>/var/log/cyberlab-bootstrap.log</code> , then rerun the bootstrap; it is safe to run twice

Running the lab on an ARM host

The DVWA and Metasploitable images are built for x86 only, so on an ARM host (an Apple Silicon Mac, or an ARM VPS such as Hetzner CAX) they fail to start until x86 emulation is turned on. Emulated targets run noticeably slower, so an x86 host is still the better choice for a class. These steps were confirmed on a Kali VM on an ARM host in September 2026.

1. **Confirm the cause.** An ARM host prints `aarch64`, the log shows `exec format error`, and the image inspect shows `amd64` (DVWA) or `386` (Metasploitable):

```
uname -m
sudo docker logs --tail 20 metasploitable
sudo docker image inspect -f '{{.Architecture}}' tleemcjr/metasploitable2
vulnerables/web-dvwa
```

2. **Register the amd64 and i386 emulators and restart the targets.** Both are needed: `amd64` for DVWA, `i386` for Metasploitable.

```
sudo docker run --privileged --rm tonistiigi/binfmt --install amd64,386
ls /proc/sys/fs/binfmt_misc/ | grep -E 'i386|x86_64' # expect qemu-i386 and
qemu-x86_64
sudo docker restart dvwa metasploitable
```

3. **Make emulation survive reboots.** The step above lasts only until the next reboot. On newer Ubuntu and Kali, `qemu-user-static` is a virtual package, so install the package that provides it:

```
sudo apt install -y qemu-user-binfmt
sudo reboot
# after the reboot, this should still list qemu-i386 and qemu-x86_64
ls /proc/sys/fs/binfmt_misc/ | grep -E 'i386|x86_64'
```

4. Pin the platforms in the lab file so a rebuild pulls the right images. In

`/opt/cyberlab/docker-compose.yml`, add a `platform:` line inside each service, at the same indentation as its `image:` line (spaces, not tabs). If a service already has a `platform:` line, change it instead of adding a second one.

```
dvwa:
  image: vulnerables/web-dvwa:latest
  platform: linux/amd64
  ...
metasploitable:
  image: tleemcjr/metasploitable2:latest
  platform: linux/386
  ...
```

Then validate the file and apply the change:

```
cd /opt/cyberlab
sudo docker compose config --quiet && echo "YAML OK"
sudo docker compose up -d dvwa metasploitable
```

5. Confirm the targets stay up. A status of "Up Less than a second" can be a container caught mid-restart, so wait and check the restart count twice; it must stop rising:

```
sleep 30; sudo docker ps --format 'table {{.Names}}\t{{.Status}}' | grep -E
'dvwa|metasploitable'
sudo docker inspect -f '{{.Name}} restarts={{.RestartCount}}' dvwa
metasploitable
sudo docker exec attacker curl -s -o /dev/null -w '%{http_code}\n'
http://10.20.0.12    # 200 or 302
```

Under emulation, DVWA's database takes 20 to 60 seconds to start and Metasploitable about a minute, so wait before scanning or exploiting. If Metasploitable starts without a format error but then exits quietly, add `tty: true` and `stdin_open: true` to its service in the compose file.

Safety rules

1. Never publish a target port. The only published ports are 127.0.0.1:9443 and 127.0.0.1:1080; keep them bound to 127.0.0.1.
2. On cloud hosts, allow inbound SSH only, and only from your IP.

3. Never run Atomic Red Team or exploit tools on the host itself; use the attacker container.
4. Do not point lab tools at anything outside 10.10.0.0/24, 10.20.0.0/24 and 10.30.0.0/24.
5. Portainer controls Docker, which means root on the host; keep it off the lab networks.
6. Stop, park or destroy cloud instances when the term ends, so no vulnerable host keeps running unattended.

Status of these scripts

Version 0.9 (September 2026). All scripts pass bash syntax checks and ShellCheck, the lab and Guacamole Compose files validate, the hcloud commands were checked against hcloud 1.69, and the single-host nftables rules pass `nft -c`. The kit has not yet been run end to end on every option. Pilot each option once on your own machine or account before giving it to students, and send the maintainer any fixes.

Sources

- [AWS Academy Learner Lab – Foundational Services \(EC2 limits, regions, vockey, LabInstanceProfile\)](#)
- [Amazon Lightsail pricing](#)
- [Hostinger KVM 4 plan specs and pricing \(VPSBenchmarks\)](#)
- [WebSploit Labs by Omar Santos](#)
- [Atomic Red Team \(Red Canary\)](#)
- [Invoke-AtomicRedTeam](#)

Hetzner Cloud:

- [Hetzner Cloud cost-optimized plans \(CX, CAX specs and locations\)](#)
- [Hetzner Cloud pricing tracker, September 2026 \(CX33 €8.49/month, €0.0136/hour\)](#)
- [Hetzner cloud pricing after the 2026 increases \(IPv4 €0.50/month, EU-only CX plans\)](#)
- [hcloud CLI](#)

Prices were checked on 26 September 2026 and change often, especially VPS promotional prices.